

The order of functions

- The speed that an algorithm can be executed depends mainly on the size of its input.

The order of functions

- Some problems are theoretically impossible to program. Other problems take interminably long.
- An $O(2^n)$ problem takes too long to run to completion using a super fast nanosecond computer that runs $O(1)$ problems in one nanosecond when n is as small as 100

$$\begin{aligned}2^{100}\text{ns} &= (2^{10})^{10}\text{ns} = 1024^{10}\text{ns} \\&\approx 1000^{10}\text{ns} \\&= 10^{30}\text{ns} = 10^{30-9}\text{seconds} = \frac{10^{21}}{3600}\text{hours} = \frac{10^{21}}{86400}\text{days} \\&= \frac{10^{21}}{31557600}\text{years} = \frac{10^{21-8}}{0.31557600}\text{years} \\&\approx 3 \times 10^{13}\text{years}\end{aligned}$$

$$2^{100}\text{ns} = 3.16681 \times 10^{13}\text{years}$$

- A *Julian year* has exactly 31557600 seconds or 365.25 days.

The fastest transistors

- A picosecond computer will take about 3×10^{10} years to do our 2^{100} problem. Feng and Hafez (2005) built a 604Ghz transistor that switches in about 2 picoseconds.
- Using a picosecond computer our problem will still be infeasible, since it will take about 3×10^{10} years. Feng and Hafez built a 604Ghz transistor in 2005. They switch at about 2 picoseconds.
- William Snodgrass, Milton Feng and Haled Hafez have improved the speed of their transistors to 845Ghz operating at $-55^{\circ}C$. At room temperature $25^{\circ}C$ it runs at 765Ghz. At the University of Illinois at Urbana-Champaign (2006, December 13). World's Fastest Transistor Approaches Goal Of Terahertz Device. ScienceDaily. Retrieved October 8, 2009, from [http://www.sciencedaily.com /releases/2006/12/061212091344.htm](http://www.sciencedaily.com/releases/2006/12/061212091344.htm)

The fastest transistors—experimental in 2011

- On 7 April 2011 Dr Yu-Ming Lin, an IBM researcher said that IBM showed its fastest graphene transistor capable of executing 0.155×10^{12} cycles per second, which is about 50% faster than predecessor chips in IBM.
- The graphene transistor has a frequency of 155 GHz. Its predecessor could achieve 100 GHz by IBM in February.

The fastest transistors—still experimental in 2014

- With a frequency of 845 gigahertz, their latest device is approximately 300 gigahertz faster than transistors built by other research groups, and approaches the goal of a terahertz device.
- LOS ANGELES, May 19, 2009—The Defense Advanced Research Projects Agency (DARPA) has awarded Northrop Grumman Corporation (NYSE: NOC) phase 1 of the \$37-million Terahertz (THz) Electronics contract, U.S. Department of Defense and company officials announced today. Northrop Grumman-developed transistor that holds the Guinness World Record for speed. The transistor provides much higher frequency and bandwidth capabilities. The fastest transistor based on a device that has a maximum frequency of oscillation well in excess of 1,000 GHz

The fastest transistors—production in 2014

- The fast “going rate” for production chips in September 2014 remains at about 3.5–4.9 Ghz.
- Intel’s June 2013 Haswell 4.5–4.9 Ghz chip has a load bandwidth of 64 bytes/cycle and a store bandwidth of 32 bytes/cycle.
- You can pick up a Core i7-4770K for \$339-00—about R3650-00 from your chip retailer.

Times for $O(2^n)$ calculations

Suppose a problem has time complexity $O(2^n)$.

Given n how long will a nanosecond machine take to execute such a problem?

n	2^n	Time taken
100	$2^{45} \times 24 \times 60 \times 60 \times 365.25 \times 10^9$	3.2×10^{13} years
95	$2^{40} \times 24 \times 60 \times 60 \times 365.25 \times 10^9$	10^{12} years
85	$2^{30} \times 24 \times 60 \times 60 \times 365.25 \times 10^9$	milliard years
75	$2^{20} \times 24 \times 60 \times 60 \times 365.25 \times 10^9$	million years
65	$2^{10} \times 24 \times 60 \times 60 \times 365.25 \times 10^9$	1000 years
55	$24 \times 60 \times 60 \times 365.25 \times 10^9$	Julian year
51	$24 \times 60 \times 60 \times 365.25/12 \times 10^9$	month
46	$24 \times 60 \times 60 \times 10^9$	day
42	$60 \times 60 \times 10^9$	hour
36	60×10^9	minute
30	10^9	second

Efficient algorithms and tractability

Ordering functions according to their growth rate.

- In a sense we cannot hope to compute these exponential problems for large n .
- *Algorithms*—or programs—that are $O(n^k)$ or $O(P(n))$ where $P(n)$ is a polynomial, are called **efficient**.
- *Problems* that yield these programs are said to be **tractable**.
- This seems to be a weird definition because a program that runs in time n^{10} is not that efficient, but neither is the *linear* problem that runs in time $10^{10}n$.
- Most tractable problems tend have practical solutions, i.e. the size of k seldom exceeds 4. They are seldom worse than quadratic.
- We call the set of problems with polynomially bounded algorithms, **P**.

Ordering functions according to their growth rate

- We categorize the running time of algorithms according to the time the algorithm tends to take to execute as a function of the length of its input n .
- An algorithm that executes in a time that does not depend upon the size of its input is said to run in constant time.
- Algorithms that run in constant time are also said to run in time $O(1)$.

Time and space complexity of algorithms

The time and space complexity of algorithms are measured

- $O(f(n))$ bounds functions from above.
- $\Theta(f(n))$ bounds functions from below.
- $\Omega(f(n))$ is a tight bound for functions.

An attempt at defining an algorithm

- An *algorithm* is a finite set of instructions for performing a computation or solving a problem.
- Synonyms for *algorithm* are: program, recipe, procedure, function, subroutine, ruleset, etc. The following algorithm tests for surjectivity.

```
1 def isOnto (function f: [1 .. n]  $\mapsto$  [1 .. m] ):  
2   if m > n: return false // cant be onto  
3   soFarIsOnto = true  
4   for j in range (1, m+1):  
5       soFarIsOnto = false  
6       for i in range (1, n+1):  
7           if f(i) == j:  
8               soFarIsOnto = true  
9       if !soFarIsOnto: return false  
10  return true
```

Complexity and growth of functions

Another algorithm that tests for surjectivity.

```
1 def isOnto (function f: [1 .. n]  $\mapsto$  [1 .. m] ):
2     if m > n: return false // cant be onto
3     beenHit = [False]*m+2
4     beenHit = (f(i) for i in range (1, n+1))
5     for j in range (1, m+1):
6         if !beenHit[j]:
7             return false;
8     return true;
```

Some recursive functions

A recursive function for $n!$

```
1 def factorial(n):  
2     if n <= 0: return 1  
3     return n*factorial(n-1)
```

A recursive function for the n -th Fibonacci number.

```
1 def Fibonacci(n):  
2     if n == 1: return 0  
3 elif n == 2: return 1  
4 else: return Fibonacci(n-1) + Fibonacci(n-2)
```

While these are not the best ways to compute these functions the code is very understandable.

Complexity and growth of functions

-
-
-
-

Complexity and growth of functions

-
-
-
-

Complexity and growth of functions

-
-
-
-

Complexity and growth of functions

-
-
-
-

Complexity and growth of functions

-
-
-
-